

The Middle Loop

March 13, 2026

Early in my career, I took a job as a software developer at a large consultancy. Turns out the work mostly involved fixing other people’s Access databases by right-clicking and hitting “repair”. I lasted about a month before I thanked them for the opportunity, handed in my resignation, and moved on.

The point is: what you spend your time on defines what you think about. What you think about is what you practice. And what you practice is what you build skill on. The title said “software developer” but the daily work said something else entirely.

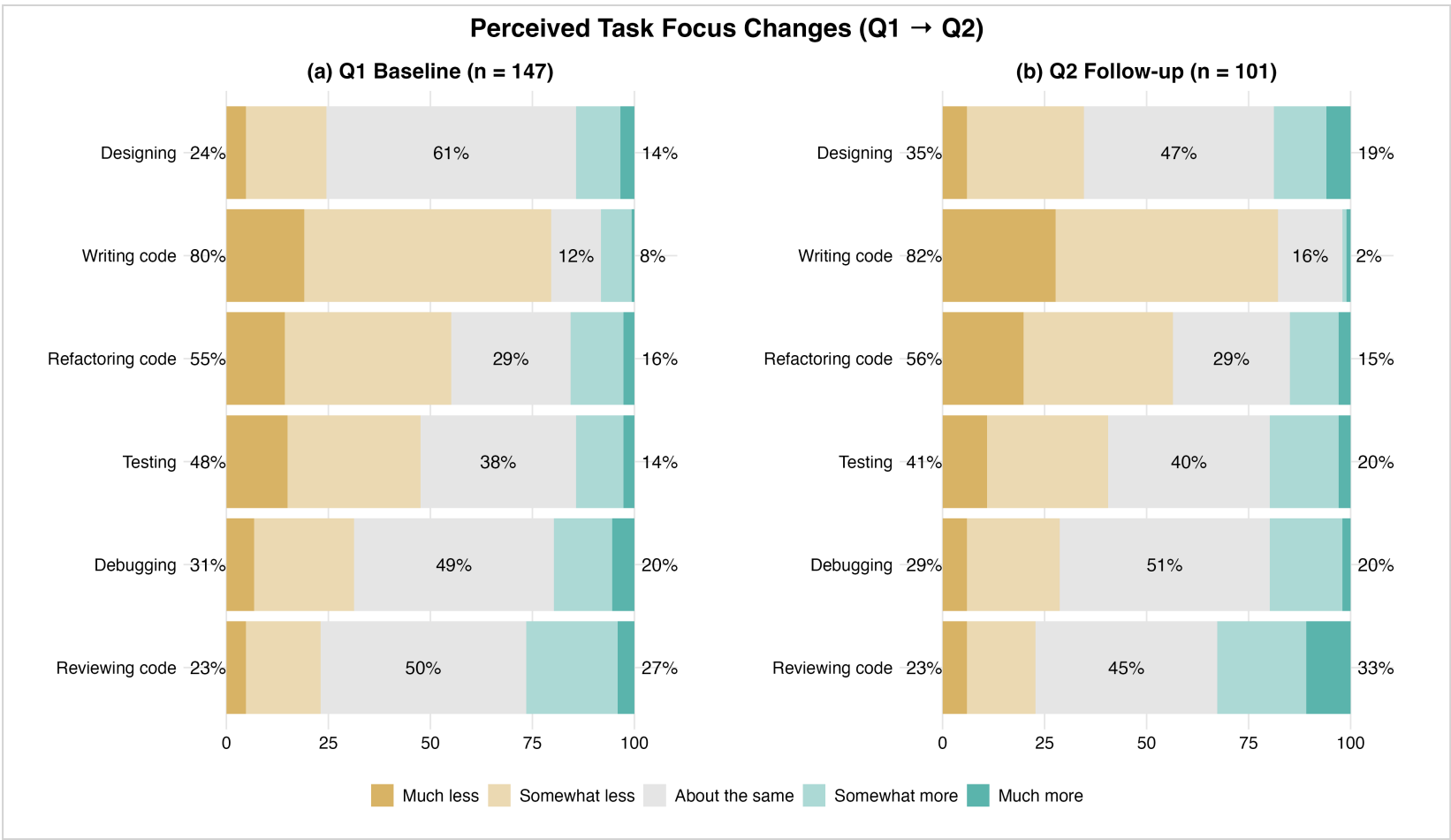
That’s why the first research question I wanted to answer as part of my Masters of Engineering at the University of Auckland, supervised by [Kelly Blincoe](#), was about task focus. Are AI tools shifting where engineers actually spend their time and effort? Because if they are, they’re implicitly shifting what skills we practice and, ultimately, the definition of the role itself.

Over six months, from October 2024 to April 2025, I ran a longitudinal mixed-methods study with professional software engineers across 28 countries - 158 eligible participants in the first round, 101 in the second, with 95 matched across both. This is the first in a series of posts sharing what I found.

The Missing Time

I asked participants how their time allocation had shifted across six core development tasks: designing, writing code, refactoring, testing, debugging, and reviewing.

Five of the six tasks showed means below neutral - participants reported spending less time on almost everything. Writing code showed the strongest reduction, with 82% reporting less time by the second questionnaire. Reviewing code was the only task where engineers reported spending *more* time, and even that was only slightly above neutral.



How participants perceived their time allocation shifted across six core development tasks. Writing code showed the largest reduction; reviewing was the only task above neutral at both timepoints.

These are perceptions, not stopwatch measurements, and people generally aren’t great at estimating time. But shifts this consistent, measured twice over six months, are still a meaningful signal.

Ok so engineers are spending less time writing code, no surprise there. But the standard assumption, that freed-up time flows upstream into design and architecture, didn’t hold. Time compressed across almost all six tasks, including design. Rather than trading writing time for design time, engineers reported spending less time on nearly everything.

So where is that time actually going?

Naming the New Work

The quantitative data revealed a **creation-to-verification shift** over the six month study period - the balance between creation-oriented tasks (writing, refactoring, designing) and verification-oriented tasks (reviewing, testing, debugging) tilted toward verification (p = 0.006, moderate effect).

But traditional categories like “reviewing” and “testing” didn’t fully capture what engineers were describing. It was the open-ended responses that told us a richer story.

“Mostly reading the code and directing AI on the right way”, explained one engineer. Another described spending more time asking *“‘is this code correct’, ‘is it maintainable’, ‘is it secure’ and ‘is this just probabilistic hallucination’”*. A third stated it quite simply: *“I ask AI to make changes first and review them before accepting or rejecting them”*.

Some participants did label this change in their workflow as “more design thinking”, but their descriptions generally told a different story - directing, evaluating, correcting. That’s supervision, a qualitatively different form of work that traditional software development lifecycle models don’t have a category for.

In my thesis, I propose a name for it: **supervisory engineering work** - the effort required to direct AI, evaluate its output, and correct it when it’s wrong.

- **Directing:** specifying intent, crafting prompts, managing context, codifying standards into reusable agent instructions and skills, and iterating when output misses the mark
- **Evaluating:** reading AI-generated output and deciding what to accept, modify, or reject
- **Correcting:** fixing errors, integrating output into existing code, and maintaining consistency across the codebase

This work is emerging because someone has to be accountable for what the AI produces, and how much of it an engineer does depends on how much they trust the AI’s output and how critical the work is.

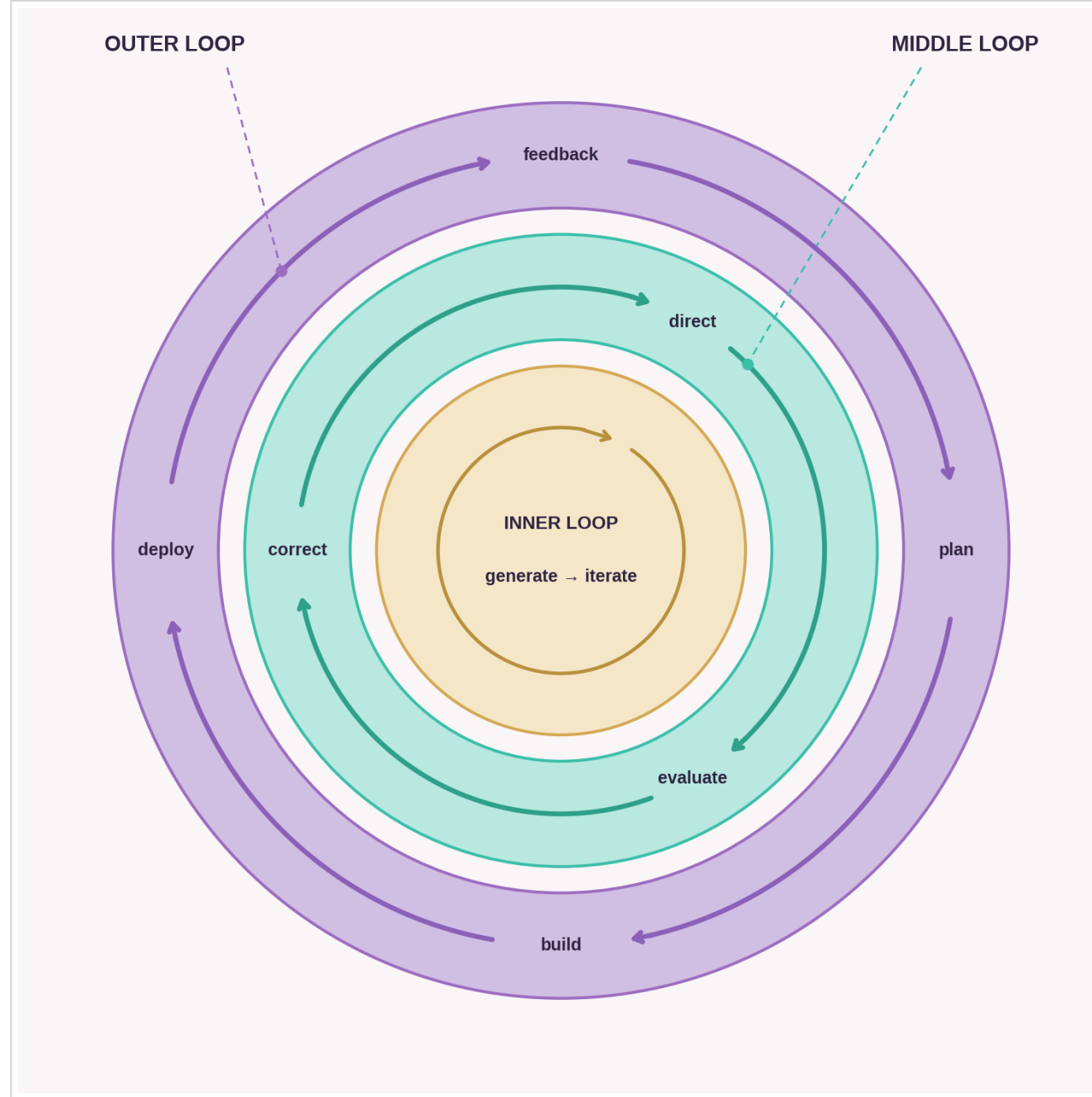
The Three-Loop Model

On the long flight to [The Future of Software Development](#) retreat in early February, I was turning this over in my head. How do you explain supervisory engineering work to practitioners in a way that maps onto their existing mental models?

It occurred to me that we often refer to “loops” in software development - the [inner loop and the outer loop](#):

- The **inner loop** is where the craft lives: write code, build, run, test, debug. Tight, fast, local. This is what TDD and better IDEs optimised.
- The **outer loop** is the broader cycle: commit, code review, CI, deploy, monitor, feedback. This is what DevOps and CI/CD optimised.

What if supervisory engineering work lives in a new loop between these two loops? AI is increasingly automating the inner loop - the code generation, the build-test cycle, the debugging. But someone still has to direct that work, evaluate the output, and correct what’s wrong. That feels like a new loop, the **middle loop**, a layer where engineers supervise AI doing what they used to do by hand.



The three-loop model: inner, middle, and outer loops

This layer is still largely unoptimised. Engineers are assembling it from chat windows, terminal agents, and IDEs that weren't designed for supervisory work. That's the frontier, and people are starting to build for it.

Birgitta Böckeler, who was also at the retreat, recently described what she calls [harness engineering](#) - the tooling, constraints, and control systems teams build to keep AI agents in check. As she notes, OpenAI's own team has said: "Our most difficult challenges now center on designing environments, feedback loops, and control systems." Steve Yegge's [Gas Town](#) project takes a different angle, building infrastructure for engineers to verify what agents produce.

Several concepts are circling around this space. Context engineering, which I've [written about previously](#), describes the practice of shaping what AI sees and works with. Harness engineering describes the infrastructure you build to constrain and guide it. Supervisory engineering work describes the activity itself - the directing, evaluating, and correcting that engineers do. The middle loop is where all of this *lives*: the structural layer between the inner and outer loops where that oversight happens.

I was glad to be able to raise this topic at the retreat, and even more so that it resonated with the other attendees. The [retreat's published findings report](#) adopted the middle loop as what it called "the retreat's strongest first-mover concept", noting that "nobody in the industry has named this yet". I won't pretend it didn't feel pretty good to see my contribution land so strongly.

What It Actually Feels Like

The experience of working in this layer is uneven. For some engineers, AI coding assistants compress the effort-to-reward loop to seconds - prompt, result, prompt, result - and it feels exhilarating. For others, the same compression is disorienting, the craft they spent years building reduced to reviewing someone else's output. Most are somewhere in between, figuring it out as they go.

The positive side is real. 84% of the participants in my study reported productivity gains. Engineers described completing in hours what used to take days, feeling more confident tackling unfamiliar tools and languages, and spending less time on the mechanical grind. As one put it: *"boring mechanical tasks like refactoring files, updating tests, adding translations, creating test fixtures are 10x less effort with AI assistants"*.

But software engineering has always offered something beyond productivity - tight feedback loops where you write code, run tests, refactor, and see *your own work* succeed. Those loops provide constant micro-rewards: the dopamine hit of a test finally passing after an hour of frustration, the pride of "I figured this out myself". That's what kept many of us hooked from the very beginning.

Supervisory work replaces those signals with something more ambiguous. You're reading more than writing. You're evaluating output that's mostly right but sometimes wrong in subtle, dangerous ways. The code works, but you didn't write it. The test passes, but you didn't wrestle with the logic. It feels like work, but it doesn't quite feel like *your* work.

And to that end, the data showed that the proportion of participants reporting negative developer experience nearly doubled over six months, rising from 14% to 27%, even as productivity held steady. I call this the **productivity-experience paradox**, but that's the subject of the next post.

Why This Matters

A lot of software engineers right now are feeling genuine uncertainty about the future of their careers. What they trained to do, what they spent years upskilling in, is shifting - and in many ways, being commoditised. The narratives don't help: either AI is coming for your job, or you should just "move upstream" into architecture and "higher value" work. Neither tells you what to actually do on Monday morning.

That's why this matters. There is still plenty of engineering work in software engineering, even if it looks different from what most of us trained for. Supervisory engineering work and the middle loop are one way of describing what that different looks like, grounded in what engineers are actually reporting.

Will the middle loop itself get compressed away? Maybe. Things are moving fast, and every abstraction layer in computing's history eventually compressed the one before it. But right now, this is where the work is, and the skills it demands (judgement, verification, accountability) transfer upward regardless of what the work looks like next year.

If we can describe it, we can start to work out what good looks like. What skills does it require? How do you get better at it? What does a career path look like when the work shifts from creation to verification, from writing to supervision? These are practical questions, and they need practical answers, but they're hard to answer when the work doesn't have a name.

What you spend your time on is what you practice, and what you practice is what you build skill on. If the work is shifting to supervision, then that's the craft now. We're building the machine that builds the machine. And it deserves the same investment in tooling, practice, and professional development that we gave the inner and outer loops before it.

What Comes Next

This post is the first in a series drawing on my Masters research and the ideas emerging from it.

Coming next: the **productivity-experience paradox** - when productivity holds steady but developer experience erodes, and what that means for the people doing the work.

A paper based on this research has been submitted for review, and I'll be speaking about these ideas at the [IT Revolution Enterprise AI Summit](#) in April and as a keynote at [Web Directions AI Engineer](#) in June.

[< Newer](#) [Older >](#)